AI Engineering and Claude Architect · Part IV, Java

AI in Java, Lesson 6

Async AI: Kafka, Batches and Workflows

A nightly job summarized 50,000 support tickets by calling the API in a loop. It hit rate limits, restarted, paid for the first 20,000 again, and starved the live assistant, which shared its quota. The Batch API would have done it for half the price, with no loop at all.

Module	AI08 of 13
Lesson	6 of 8
Prerequisites	AI03 L5, AI08 L1, HLD on Kafka
Exam relevance	**TS 4.5** (batch processing). The Java itself is out of scope
Reading time	~30 minutes

Contents

1. Async AI

Lesson 6 of 8 · AI in Java

Async AI

> *Key idea*
>
> **If nobody is waiting, don't make the work synchronous.** Choose by how long a result can wait: Kafka for near-real-time events, the Batch API for hours-long bulk work at half price, and a workflow engine for multi-step processes. **Delivery is at-least-once, so every consumer must make a duplicate harmless, and only retry what could succeed.**

> *Before: errors and retries (AI08 L1), Batch basics (AI03 L5). Teaches: a Kafka worker with deduplication and dead letters, a Batch job in Java, and how to choose. Used by: AI08 L8, AI10.*

The hook: a real broker

A real embedded Kafka broker in the test JVM, with a scripted model. Real output from `ReviewWorkerTest`:

```
3 events, 2 unique: model calls=2 e1=Summary of first ticket
transient failure: attempts=3, then dead letter: e3|always-busy ticket
permanent failure: attempts=1, then dead letter: e4|bad-request ticket
```

> *Key idea*
>
> **A duplicate cost nothing, a busy provider got 3 tries, and a bad request got 1.** Without those decisions, duplicates pay twice, and bad requests retry forever.

Multiplying retries (AI08 L1). This test uses a scripted model, so each attempt is one call. With the real SDK and `maxRetries` of 2, 3 attempts would mean up to 9 API requests. Decide where retries live.

Intuition: three ways to get things done

Option	Everyday version	Good for
Sync call	Asking across the desk	A person is waiting
Kafka	An office in-tray, worked through in order	Minutes, steady flow
Batch API	Bulk mail at a discount, delivered by tomorrow	Hours, large volumes
Workflow engine	A case file moving through departments, over days	Multi-step, with waits

Mechanism: choosing

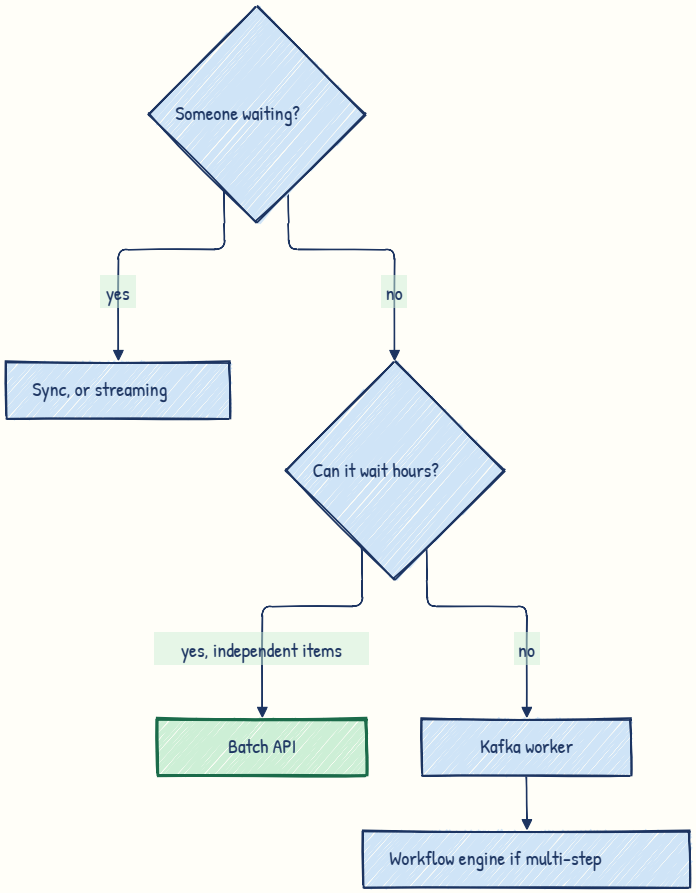


Figure 1. Choosing a shape. The deciding questions are whether someone's waiting, how long the result can wait, and whether there are several steps.

Factor	Sync	Kafka	Batch API	Workflow engine
Latency	Seconds	Seconds to minutes	**Up to 24 hours**	Any
Cost	Standard	Standard	**50% less**	Standard
Multi-turn tools	Yes	Yes	**No**	Yes
Rate limit pressure	Direct	You control it	Separate handling	You control it
Blocking check before merge	Yes	Possibly	**No**	Possibly

Kafka: the consumer

From `ReviewWorker.java`. The event ID is claimed first with a unique insert:

```
try {
    jdbc.update("INSERT INTO processed_events (event_id) VALUES (?)", id);
} catch (DuplicateKeyException alreadySeen) {
    return;
}
```

And a failure releases the claim, so a retry can do the work:

```
} catch (ModelException e) {
    // Release the claim, so the retry can do the work.
    jdbc.update("DELETE FROM processed_events WHERE event_id = ?", id);
    if (!e.kind().retryable) {
        throw new KafkaErrors.PermanentFailure(e);
    }
    throw e;
}
```

From `KafkaErrors.java`, which retries twice, then sends it to a dead letter topic, skipping retries for permanent failures:

```
DefaultErrorHandler handler = new DefaultErrorHandler(
        new DeadLetterPublishingRecoverer(kafka), new FixedBackOff(200L, 2L));
handler.addNotRetryableExceptions(PermanentFailure.class);
```

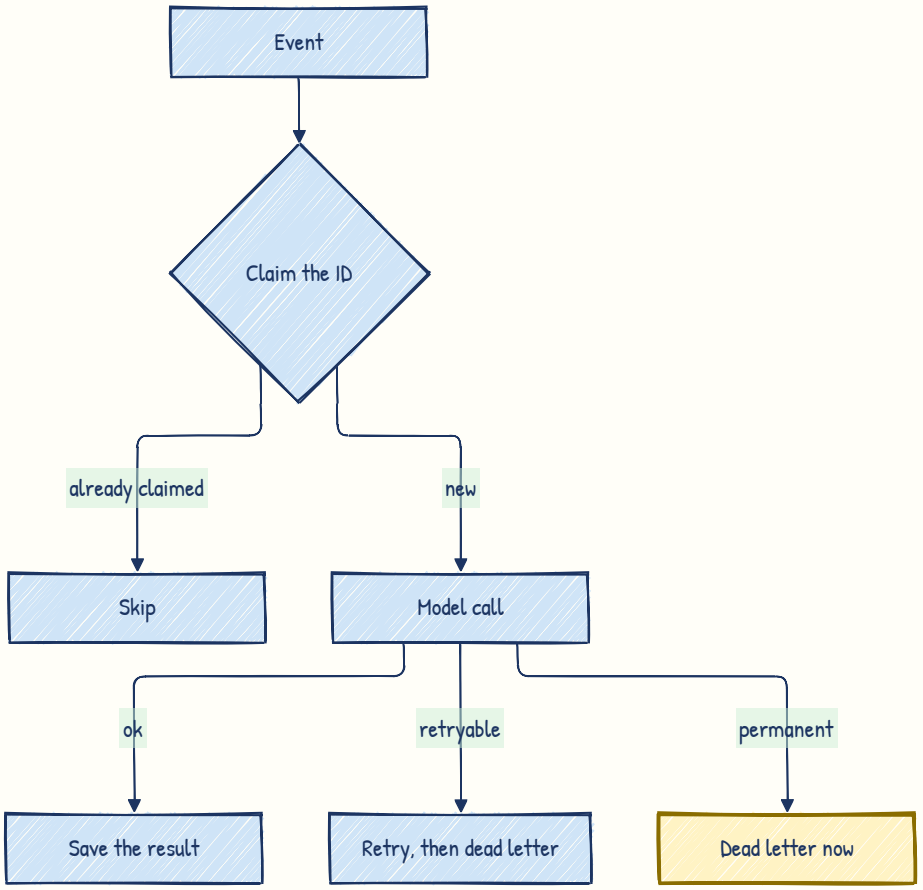


Figure 2. An event's possible paths. Duplicates exit early, transient failures retry, and everything unresolved ends up on the dead letter topic, where someone can see it.

Weakness in this sketch	Effect	Fix
A crash after claiming	The event is stuck as "claimed", and is skipped forever	Claim with a timestamp, and let claims expire
Fixed 200 ms backoff	Too quick for an overload	Exponential, with jitter
A busy API blocks the partition	Others wait	Pause the consumer, or use retry topics
No rate limit	Can compete with live traffic	Limit concurrency, or use a separate workspace

Batch API: the job

Batch facts (TS 4.5): 50% cheaper, up to 24 hours, no latency guarantee, no multi-turn tool loops, and results in **any order**, so match by `custom_id`, and resubmit only failures.

From `BatchJob.java`, on the real SDK against the fake API. Real output, where results arrived as b, c, a:

```
batch id: batch_1, path /v1/messages/batches, requests sent: 3
ended? true
results path: /v1/messages/batches/batch_1/results
outcome (arrived b, c, a): Outcome[succeeded={doc-b=Summary B, doc-a=Summary A}, toResubmit=[doc-c]]
```

```
if (item.result().isSucceeded()) {
```

```
succeeded.put(item.customId(), text.toString());
```

Each result carries its `custom_id`, so no position assumptions are made.

The cost of the hook's job, on Sonnet 5.5 at $2 / $10, with 1,500 input and 150 output tokens per ticket:

Approach	Calculation	Cost
Sync loop, 50,000 tickets	50,000 × $0.0045	**$225**
Same, with 20,000 repeated after the restart	70,000 × $0.0045	**$315**
Batch API	50,000 × $0.00225	**$112.50**

These are illustrative token counts. Prompt caching can also apply within batches, though cache hits there are less predictable.

Workflow engines

For several steps with long waits (AI07 L2), a durable engine keeps state, retries and timers. Temporal is one for Java, stated from general knowledge. **I haven't run one here.** The simpler alternative is a status column and a scheduled job, which is fine for one or two flows.

Why the obvious fixes fall short

Tempting move	Why it fails	Better
A sync loop with `sleep`	Slow, full price, and loses progress	Batch API
`@Async` with no queue	Lost on restart	Kafka or a durable queue
Exactly-once delivery as a goal	Hard, and external calls aren't covered	At-least-once, with idempotent work
Retry everything	Bad requests retry forever	Classify, then dead letter
Batch for pre-merge checks	No latency guarantee	Sync
Batch for multi-turn tools	Not supported	Kafka worker running a loop

How Anthropic approaches it

Point	Source
Batches run at 50% cost, asynchronously	Claude API reference, AI03 L5
Results come in any order, so key by `custom_id`	Same
Workspaces separate limits. Separate batch from live	AI08 L1
When work fans out across hundreds of documents, Managed Agents has dynamic workflows	Claude API reference

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: the sync loop.

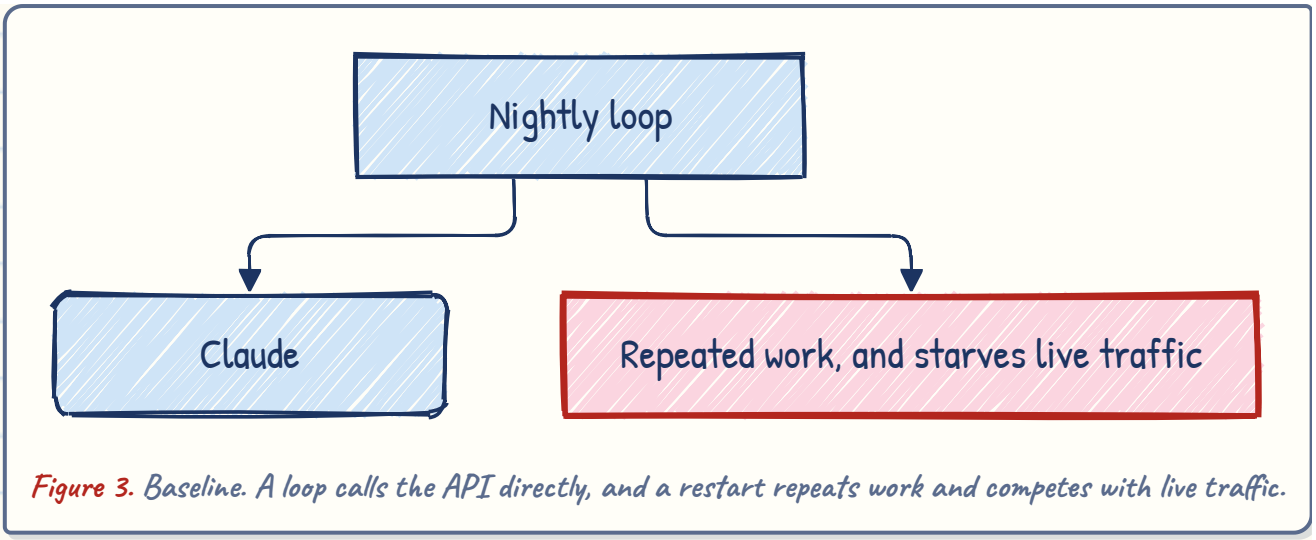


Figure 3. Baseline. A loop calls the API directly, and a restart repeats work and competes with live traffic.

Improved: Kafka workers. Durable, with retries. New problem: **at-least-once delivery means duplicates, and a burst of work can use up the quota that live users need.**

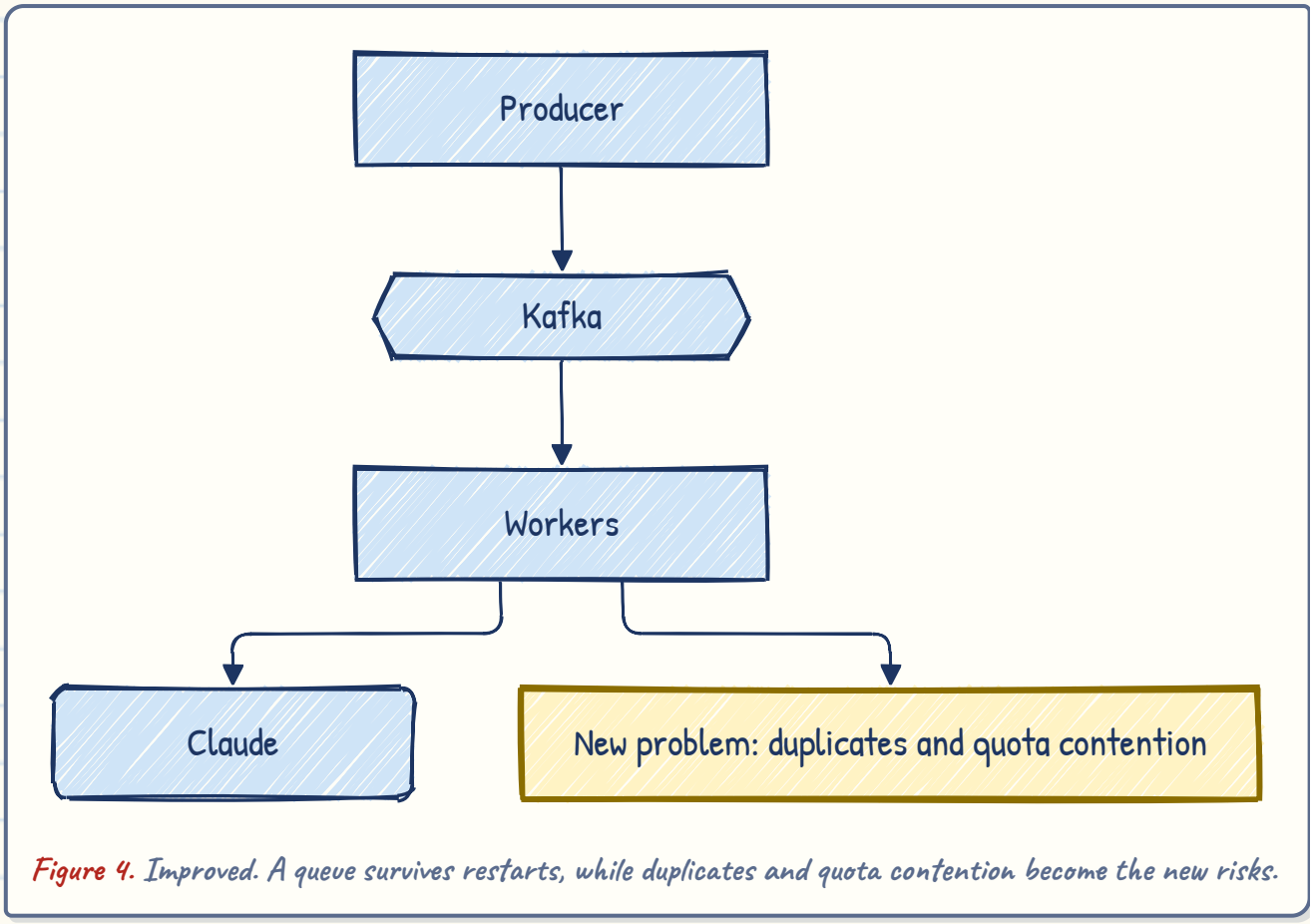


Figure 4. Improved. A queue survives restarts, while duplicates and quota contention become the new risks.

Production: right tool per job, with separate limits.

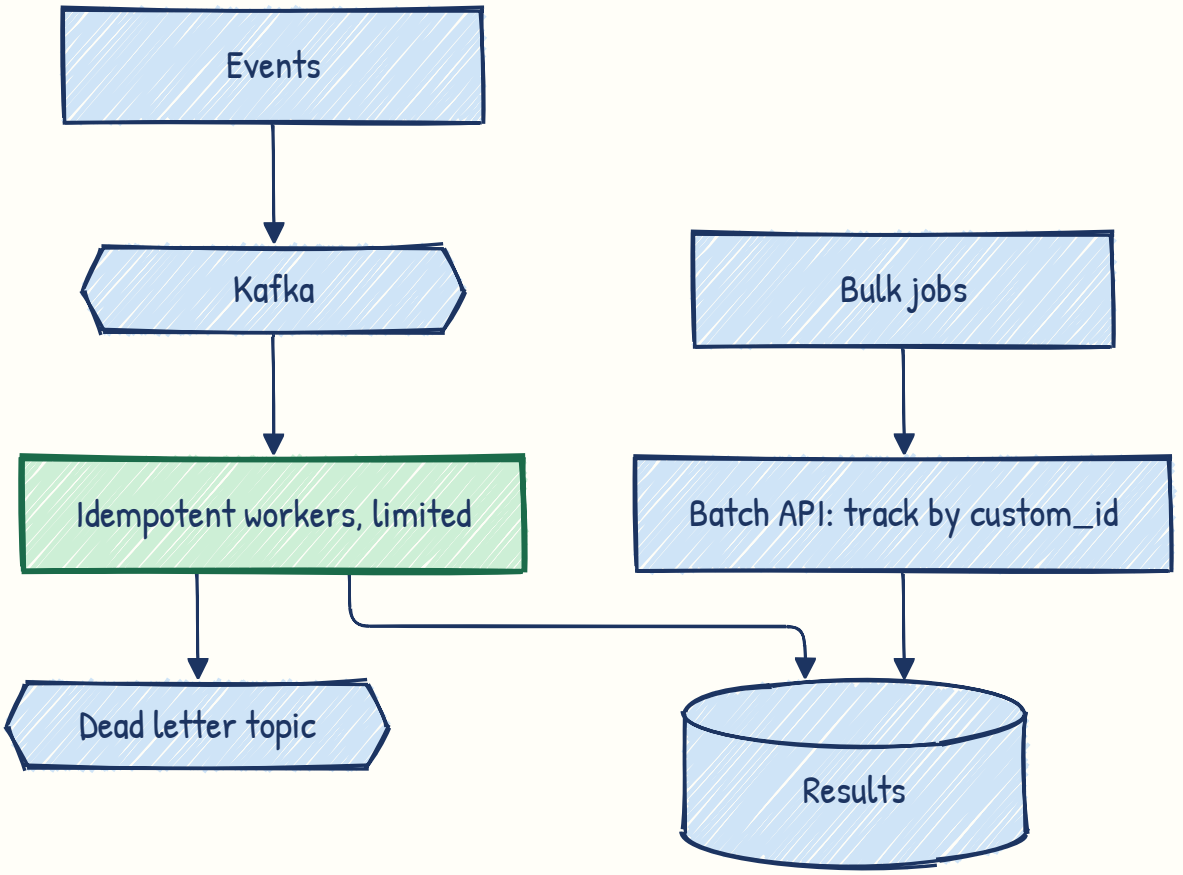


Figure 5. Production. Near-real-time work goes through Kafka with idempotent workers, bulk work goes to the Batch API, and each uses its own workspace, so live traffic is protected.

Component	Why it exists	How it communicates
Partition key	Keeps related events in order	Set by case or document
Dedup table	Duplicates are harmless	Unique key, with expiry
Dead letter topic	Nothing is lost silently	Alert and a replay tool
Batch tracker	Maps `custom_id`s, and resubmits failures	Database table
Separate workspace	Bulk can't starve live	Separate API key

Failures

Failure	Containment
Duplicate delivery	Claim by ID
Poison message	Non-retryable, so dead letter
Worker crash after claim	Expiring claims
Batch partly fails	Resubmit only those `custom_id`s
Batch takes too long	Fall back, or alert at a deadline
Dead letter topic grows	Alert, and read it. It's often a new bug

Trade-offs

Decision	Gain	Cost
Batch API	Half price, no rate juggling	Slow, no multi-turn tools
Kafka	Durable, near-real-time	Infrastructure, duplicates
Workflow engine	Long processes	Another system to run
Dedup table	Safe retries	A write per event

When simple is fine: a low-volume background task can use a database table and a scheduled job.

Common mistakes

1. **Looping sync calls for bulk work.** Use batches.
2. **Using position to match batch results.** Use `custom_id`.
3. **Retrying the whole batch.** Resubmit only failures.
4. **Batch for blocking checks.** There's no latency guarantee.
5. **Assuming no duplicates.** Delivery is at-least-once.
6. **Retrying permanent failures.** Send them to the dead letter topic.
7. **Claiming with no expiry.** A crash strands the event.
8. **Sharing quota with live traffic.** Use separate workspaces.

What to say in an interview

"I'd choose by how long a result can wait. Hours and independent items go to the Batch API at half price, matched by custom_id, with only failures resubmitted. Near-real-time goes through Kafka, with idempotent consumers, since delivery is at-least-once, and permanent failures go straight to a dead letter topic. Bulk work gets its own workspace, so it can't starve live traffic."

Certification lens

How this lesson maps to CCAR-F

TS 4.5. Batch is 50% cheaper, up to 24 hours, with no latency guarantee and no multi-turn tools. Correlate by `custom_id`, and resubmit only failures. A common wrong answer uses it for blocking pre-merge checks.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Claude Code for CI/CD · Medium. Which job suits the Message Batches API best?

A. A pre-merge check that blocks a pull request until complete.

B. A nightly report summarizing yesterday's merged changes.

C. An agent that makes several tool calls to investigate a failure.

D. A chat assistant for developers.

A **B** (TS 4.5). A needs a latency guarantee. C needs multi-turn tools. D needs real-time replies.

Q Q2 · Exam style, scenario: Structured Data Extraction · Medium. A batch of 10,000 documents returns with 230 failures. What's the best way to continue?

A. Resubmit the whole batch.

B. Process the rest by position in the original list.

C. Ignore failures, as 97.7% succeeded.

D. Identify failures by `custom_id`, and resubmit only those, adjusting anything that caused them.

A **D** (TS 4.5). A pays twice. B is wrong, since results arrive in any order. C silently loses data.

Q Q3 · Calculation · Medium. 200,000 documents on Sonnet 5.5 ($2 / $10), each 2,000 input and 200 output tokens. What are sync and batch costs? What if the deadline is 2 hours?

A Per document: $0.004 + $0.002 = $0.006. Sync: **$1,200**. Batch: **$600**. Batches can take up to 24 hours, so a 2-hour deadline can't rely on them. Use workers with limited concurrency, and check that rate limits allow it.

Revision - Lesson 6

→ Nobody waiting means don't work synchronously.

→ Sync for people, Kafka for minutes, Batch for hours, workflows for many steps.

→ Batch: 50% cheaper, up to 24 hours, no multi-turn tools, any order.

→ Match by custom_id, and resubmit only failures.

→ Not for blocking checks.

→ Kafka is at-least-once, so claim by event ID.

→ Release a claim on failure, and let claims expire.

→ Retry only what could succeed, then dead letter.

→ Retries multiply with SDK retries.

→ Separate bulk from live traffic with workspaces.

Your notes